

FINAL PROJECT REPORT

**Design and FPGA Implementation
of a Single-Cycle RISC-V SoC with
Wishbone Bus
and Memory-Mapped LED Pattern Controller**

Student Name	Hassan Ali Ahmad
Student ID	NUM-BSEE-2022-03
Course Instructor	Dr. Tassadaq Hussain
Submission Date	16 June 2026

Abstract

This report presents the design, verification, debugging, and FPGA implementation of a single-cycle RV32I System-on-Chip (SoC) with a Wishbone-based memory and peripheral interface. The project implements a custom RISC-V processor in Verilog HDL and integrates it with instruction memory, data memory, an address decoder, a Wishbone interconnect, and a memory-mapped LED peripheral. The LED peripheral is mapped at address 0x00002000, allowing bare-metal firmware running on the processor to control the FPGA LED outputs through ordinary store instructions.

The project uses a complete hardware/software co-design flow. On the software side, a C firmware program writes the LED pattern sequence 0x01, 0x02, 0x04, 0x08, 0x04, and 0x02 repeatedly to the memory-mapped LED address. The firmware is compiled for the RV32I instruction set using the RISC-V GCC toolchain, linked at the reset address 0x00000000, converted into a raw binary and hexadecimal program file, and then transformed into a Verilog ROM initialization include file. On the hardware side, the processor fetches instructions from the Wishbone ROM, executes arithmetic, branch, load/store, and jump operations, and performs Wishbone transactions to memory and the LED slave.

The final project files also include fixes required for reliable hardware behavior. These fixes include a hardware-safe startup reset, corrected reset propagation to the processor and register file, cleaned ROM/RAM/LED acknowledgement behavior, a larger firmware delay loop for visible FPGA LED blinking, separated LPF constraint files for supported boards, and a hardware-only blinker test for validating clock and LED pin mappings before running the full SoC. The implementation flow uses open-source tools, including Icarus Verilog and GTKWave for functional verification, Yosys for ECP5 synthesis, nextpnr-ecp5 for placement and routing, ecppack for bitstream packing, and openFPGALoader for programming. The report provides image placeholders so that simulation screenshots, terminal outputs, FPGA programming logs, and physical LED photographs can be inserted after final testing.

Table of Contents

Abstract	i
1 Introduction	1
1.1 Project Overview	1
1.2 Motivation	1
1.3 Report Construction Approach	1
1.4 Scope of the Project	2
1.5 Main Contributions	2
2 Problem Statement and Project Objectives	2
2.1 Problem Statement	2
2.2 Project Objectives	3
2.3 Design Requirements	3
2.4 Expected Deliverables	4
3 Background and Literature Review	4
3.1 RISC-V Architecture	4
3.2 Single-Cycle and Multi-State Educational Processor Design	5
3.3 Wishbone Bus Protocol	5
3.4 Open-Source FPGA Toolchain	5
3.5 Simulation Tools	6
4 Design Methodology	6
4.1 Overall Development Strategy	6
4.2 Design Flow	6
4.3 Hardware/Software Boundary	7
4.4 Verification Method	7
4.5 Reason for Keeping Screenshot Placeholders	8
5 System Architecture	8
5.1 Top-Level System Description	8
5.2 Top-Level SoC Block Diagram	8
5.3 Implemented Module Summary	8
5.4 Data Flow	10
5.5 Address Regions	10
6 RV32I Processor Core Design	10
6.1 Processor Organization	10
6.2 Instruction Field Extraction	10
6.3 Control Unit	11
6.4 Immediate Generator	11
6.5 Register File	12

6.6	ALU and ALU Control	12
6.7	Branch and Jump Logic	12
6.8	Processor State Machine	12
6.9	Load and Store Alignment	13
7	Wishbone Interconnect and Memory Map	13
7.1	Wishbone Master Interface	13
7.2	Address Decoder	13
7.3	Read Data Multiplexing and Acknowledgement	14
7.4	Program ROM	14
7.5	Data RAM	14
7.6	LED Peripheral Region	14
7.7	Memory Layout in Linker Script	15
8	Firmware and ROM Initialization Flow	15
8.1	Bare-Metal Firmware Purpose	15
8.2	LED Pattern Sequence	15
8.3	Startup Code	16
8.4	Compiler and Linker Flow	16
8.5	Binary, HEX, and ROM Include Generation	16
8.6	Hand-Assembled Firmware Reference	17
9	Memory-Mapped LED Peripheral	18
9.1	LED Slave Architecture	18
9.2	Wishbone Transaction Behavior	18
9.3	LED Register Logic	18
9.4	Software-to-Hardware LED Path	19
9.5	Human-Visible Delay	19
9.6	Hardware-Only Blinker as First Test	20
10	Simulation and Verification	20
10.1	Verification Strategy	20
10.2	Testbench Role	20
10.3	Icarus Verilog Compilation	21
10.4	Waveform Inspection in GTKWave	22
10.5	Simulation-to-Hardware Gap	23
11	FPGA Implementation Flow	23
11.1	Target FPGA and Board Assumptions	23
11.2	Synthesis with Yosys	23
11.3	Place-and-Route with nextpnr-ecp5	24
11.4	Bitstream Packing with ecppack	25
11.5	Programming with openFPGALoader	25

11.6 Pin Constraint Summary	26
12 Debugging and Hardware Fixes	27
12.1 Why Hardware Debugging Was Necessary	27
12.2 Startup Reset Correction	28
12.3 Register File Reset	28
12.4 Acknowledgement Handling.....	28
12.5 Delay Loop Fix.....	28
12.6 Constraint File Separation.....	29
12.7 Debug Evidence Placeholder	29
13 Results and Discussion	29
13.1 Functional Result	29
13.2 Instruction Coverage.....	29
13.3 Hardware Build Result.....	30
13.4 Expected LED Output Evidence.....	30
13.5 Discussion of Design Quality.....	31
13.6 Resource Utilization.....	31
14 Limitations and Future Improvements	32
14.1 Current Limitations	32
14.2 Possible Future Improvements	32
14.3 Recommended Next Version.....	33
15 Conclusion	33
A Selected Source Code Listings	33
A.1 Firmware Source.....	33
A.2 Startup Code.....	34
A.3 Linker Script.....	35
A.4 Wishbone LED Peripheral.....	35
A.5 Wishbone Interconnect.....	36
A.6 Top-Level SoC Excerpt	38
A.7 ALU Source.....	45
A.8 Control Unit.....	49
A.9 Build Scripts.....	50
A.10 Constraint File	51
A.11 Hardware-Only Blinker.....	52
References	53

List of Figures

1	Design methodology flow from Verilog and C firmware to FPGA LED output.	7
2	Overall SoC system block diagram showing CPU, Wishbone interconnect, ROM, RAM, and LED peripheral.....	8
3	RISC-V processor internal datapath and finite-state control diagram.	13
4	Wishbone bus architecture diagram with address decoder, read data multiplexer, ROM, RAM, and LED slave.	14
5	Linker script memory layout showing ROM at 0x00000000 and RAM at 0x00001000.....	15
6	GCC compilation flow from C/startup/linker files to ELF, BIN, HEX, and ROM initialization include file.	17
7	Terminal screenshot showing successful firmware compilation and generated program.hex / rom_init.vh.	17
8	Screenshot showing the generated program.hex loaded into the Verilog ROM initialization process.	18
9	LED pattern flow diagram from C firmware to Wishbone write transaction and physical LED output pins.....	19
10	Testbench architecture showing clock/reset generation, SoC instance, waveform dump, and observed output signals.....	21
11	Terminal screenshot showing successful Icarus Verilog compilation and simulation execution.	21
12	GTKWave screenshot showing PC, instruction, Wishbone address/data, acknowledgement, and LED output signals.	22
13	GTKWave screenshot focused on the LED pattern values written to the memory-mapped LED peripheral.....	23
14	Terminal screenshot showing successful Yosys synthesis and generation of cpu.json.	24
15	Terminal screenshot showing successful nextpnr-ecp5 placement and routing.	24
16	Terminal screenshot showing successful ecppack bitstream generation and cpu.bit file creation.....	25
17	Terminal screenshot showing FPGA detection through CMSIS-DAP and successful bitstream upload.....	26
18	Screenshot or image of the LPF constraint file and final pin mapping used for the board.	27
19	Screenshots showing the before/after debugging evidence, especially reset, LED delay, and Wishbone acknowledgement behavior.	29
20	Physical FPGA board photograph showing the LED pattern generated by the RV32I processor firmware	31

List of Tables

1	Design requirements for the RV32I Wishbone LED SoC.	4
2	Main modules in the submitted Wishbone-based SoC.	9
3	Implemented address regions in the Wishbone interconnect.....	10
4	Instruction classes decoded by the main control unit.	11
5	Processor finite-state behavior in the submitted top-level module.....	12
6	Functional verification checklist for waveform inspection.	22
7	Pin assignments in the Colorlight i5 LPF file.....	26
8	Important debugging fixes found in the submitted fixed CA project.....	27
9	Instruction behavior required or supported by the project.	30
10	Resource utilization placeholder to be filled from final synthesis/place-and-route report.	32

1 Introduction

1.1 Project Overview

This project implements a compact but complete processor-based System-on-Chip using Verilog HDL and an FPGA implementation flow. The central design is a custom RV32I processor connected to memory and an LED peripheral through a Wishbone bus. Unlike a standalone combinational or sequential logic project, this work combines instruction execution, memory mapping, firmware generation, bus decoding, simulation, synthesis, pin constraints, and FPGA programming into one complete Computer Architecture project.

The submitted CA work contains two important parts. The first is a hardware-only LED blinker, which verifies that the FPGA clock, bitstream loading process, and LED pin mappings are correct. The second is the full Wishbone-based RV32I SoC, where the processor fetches instructions from ROM, executes a bare-metal firmware program, and controls the LED outputs through a memory-mapped peripheral. This two-stage structure is important because it separates board-level hardware verification from full CPU-level debugging.

1.2 Motivation

Computer architecture is best understood when a processor is not studied only as a block diagram but is implemented, simulated, synthesized, and tested on hardware. A single-cycle or simple multi-state RV32I implementation is educationally useful because the relationship between the program counter, instruction fields, control signals, ALU operations, memory access, and write-back path remains visible. RISC-V is especially suitable for such a project because it is an open instruction set architecture with clearly documented base instruction formats and implementation rules [1, 2].

The Wishbone bus adds a system-level layer to the processor design. Instead of connecting the CPU directly to one memory block and one LED register, the design uses a master-slave interconnect with address decoding. This reflects the organization of real embedded systems, where processors communicate with memories, timers, GPIOs, UARTs, accelerators, and other peripherals through a common bus protocol. The Wishbone specification is widely used in open-source IP designs because it defines portable master/slave bus signals and transaction behavior [3].

1.3 Report Construction Approach

The previous Word report was used as a reference for presentation style, sequence, and expected academic structure. The technical report written here is based on the actual CA project files, especially the fixed FPGA project folder. Therefore, the discussion focuses on the implemented Verilog modules, firmware files, build scripts, constraint files, debugging changes,

and expected hardware validation flow. Screenshot locations have intentionally been kept as placeholders so that the final terminal screenshots, waveforms, and board photographs can be inserted later without restructuring the report.

1.4 Scope of the Project

The scope of this project includes the following tasks:

- Implementation of a custom RV32I processor core in Verilog HDL.
- Integration of instruction memory, data memory, and an LED slave peripheral through a Wishbone interconnect.
- Development of bare-metal C firmware for memory-mapped LED control.
- Conversion of compiled firmware into `program.hex` and `rom_init.vh` for ROM initialization.
- Functional verification through simulation and waveform inspection.
- FPGA synthesis, place-and-route, bitstream generation, and programming using open-source tools.
- Debugging of hardware-only issues related to reset, acknowledgements, delay timing, and pin constraints.

1.5 Main Contributions

The main contribution of this project is an end-to-end working flow from Verilog CPU design to visible LED output on FPGA hardware. The project does not stop at simulation; it includes the practical steps required to build firmware, map memory addresses, initialize ROM, synthesize the hardware, apply LPF constraints, generate a bitstream, and program the board. This makes the project closer to a real embedded SoC implementation than a simple processor simulation exercise.

2 Problem Statement and Project Objectives

2.1 Problem Statement

A common limitation in basic computer architecture laboratories is that students often design isolated components such as adders, registers, ALUs, or instruction decoders without integrating them into a full processor-driven system. Such exercises are useful but do not show how software, memory, buses, and physical I/O interact in a working embedded system. The challenge addressed in this project is to build a processor-based SoC in which firmware

running on a custom CPU controls actual FPGA pins through a memory-mapped peripheral.

The design problem includes several practical constraints. The processor must begin execution at the correct reset address. The C program must be compiled for the implemented RV32I subset. The instruction memory must be initialized with correctly ordered 32-bit words. The Wishbone interconnect must decode addresses correctly and route acknowledgement and data signals back to the CPU. The LED peripheral must update visible hardware pins when the firmware writes to the memory-mapped LED register. Finally, the design must survive synthesis and FPGA startup behavior, not only simulation.

2.2 Project Objectives

The project objectives are listed below:

1. Design a Verilog-based RV32I processor capable of executing the firmware required for LED pattern generation.
2. Implement the main processor datapath components, including the program counter, immediate generator, register file, ALU, ALU control, and main control unit.
3. Connect the CPU to Wishbone-compatible ROM, RAM, and LED slave blocks through an address-decoding interconnect.
4. Map the LED peripheral at address 0x00002000 and expose the lower bits of the LED register to FPGA output pins.
5. Write and compile bare-metal firmware that writes visible LED patterns to the memory-mapped LED address.
6. Verify the design through simulation and waveform observation before programming the FPGA.
7. Generate an FPGA bitstream using Yosys, nextpnr-ecp5, and ecppack.
8. Program the FPGA using openFPGALoader and document the hardware test results with screenshots and photographs.

2.3 Design Requirements

The functional requirements of the design are summarized in Table 1. These requirements were derived from the submitted fixed project files and the actual firmware/hardware flow.

Table 1: Design requirements for the RV32I Wishbone LED SoC.

Requirement	Expected implementation	Evidence in CA files
Instruction execution	CPU fetches, decodes, executes, and updates PC	top.v, control_unit.v, ALU.v
Memory map	ROM, RAM, and LED slave occupy separate address regions	wb_interconnect.v, link.ld
LED control	Firmware writes to 0x00002000; LED slave captures lower bits	code.c, wb_led.v
ROM initialization	Firmware words included through Verilog ROM include file	program.hex, rom_init.vh, wb_rom.v
FPGA build	ECP5 synthesis, place-and-route, and bitstream packing	build_wishbone_colorlight_i5.sh
Board constraints	Clock, reset, and LED pins mapped in LPF file	constraints_colorlight_i5.lpf

2.4 Expected Deliverables

The expected deliverables are a complete Verilog RTL design, a compiled firmware image, a simulation waveform evidence set, a synthesizable FPGA project, final bitstream generation evidence, and physical LED output evidence. In this report, the image positions are reserved as placeholders so that the final screenshots can be inserted directly after testing.

3 Background and Literature Review

3.1 RISC-V Architecture

RISC-V is an open instruction set architecture designed around reduced instruction set computing principles. The project uses the 32-bit base integer subset, commonly referred to as RV32I. The RV32I base ISA uses fixed-length 32-bit instructions and defines 32 general-purpose integer registers. Its instruction encoding is organized into major formats such as R-type, I-type, S-type, B-type, U-type, and J-type. These formats make decoding systematic because fields such as opcode, destination register, source registers, immediate bits, function bits, and branch offsets occupy well-defined bit positions [2].

The selected design uses RV32I because it is sufficient for bare-metal embedded programs such as LED control, memory writes, loops, conditional branches, and basic arithmetic. The firmware in the project uses simple instructions such as `lui`, `addi`, `sw`, `bne`, `slli`, `sltiu`, and

jal. These instructions are enough to build a visible LED walking pattern and a delay loop without requiring multiplication, division, interrupts, privilege modes, or operating system support.

3.2 Single-Cycle and Multi-State Educational Processor Design

A pure single-cycle processor completes instruction fetch, decode, execution, memory access, and write-back within one clock cycle. Such a design is easy to describe conceptually but difficult to run at a high clock frequency because the longest instruction path determines the period. The submitted CA work uses a simple finite-state structure in top.v. The processor separates instruction fetch waiting, instruction execution, and data access waiting into states such as S_FETCH, S_WAIT_INSTR, S_EXECUTE, and S_WAIT_DATA. This structure makes Wishbone acknowledgement handling easier because instruction and data memory responses are sampled only when the selected slave asserts acknowledgement.

This approach remains educational because the core datapath is still direct and transparent. The PC is updated explicitly, the instruction register holds the fetched instruction, and the control unit derives signals from the opcode. The ALU executes arithmetic and comparison operations, while the register file provides two read ports and one write port. The design avoids pipeline hazards, forwarding, branch prediction, caches, and privilege-level complexity, making it suitable for a Computer Architecture final project.

3.3 Wishbone Bus Protocol

Wishbone is an open SoC interconnection specification that defines master-slave bus transactions using signals such as address, data, write enable, select, cycle, strobe, and acknowledgement [3]. In this project, the CPU behaves as the Wishbone master. The ROM, RAM, and LED peripheral behave as Wishbone slaves. The interconnect decodes the high address bits and routes one transaction to one selected slave at a time.

The useful feature of Wishbone in this project is modularity. The LED peripheral does not need to know the internal CPU datapath. It only needs to respond to Wishbone write transactions addressed to its region. Similarly, ROM and RAM expose a simple bus interface. As a result, the system can be extended later by adding a UART, timer, seven-segment display controller, or GPIO block with a new address region.

3.4 Open-Source FPGA Toolchain

The implementation flow uses an open-source ECP5 FPGA toolchain. Yosys performs Verilog synthesis and includes an ECP5 synthesis command intended for Lattice ECP5 FPGA targets [4]. nextpnr provides open-source place-and-route support for multiple FPGA families, including Lattice ECP5 through Project Trellis support [5, 6]. ecppack converts the routed

configuration into a bitstream format, and openFPGALoader is used to upload the bitstream to the FPGA board [7].

This toolchain is especially valuable in an educational setting because it allows the complete design flow to be performed without proprietary synthesis or programming tools. The report therefore documents not only the RTL architecture but also the firmware build, synthesis command, constraint mapping, bitstream generation, and programming sequence.

3.5 Simulation Tools

Before FPGA programming, the design must be verified in simulation. Icarus Verilog compiles and runs Verilog simulations, while GTKWave allows the generated VCD waveform to be inspected visually [8, 9]. Simulation is essential because internal signals such as PC, instruction, Wishbone address, acknowledgement, ALU result, register write-back, and LED output can be observed before hardware deployment. FPGA testing confirms visible behavior, but waveform simulation helps identify why a behavior occurs.

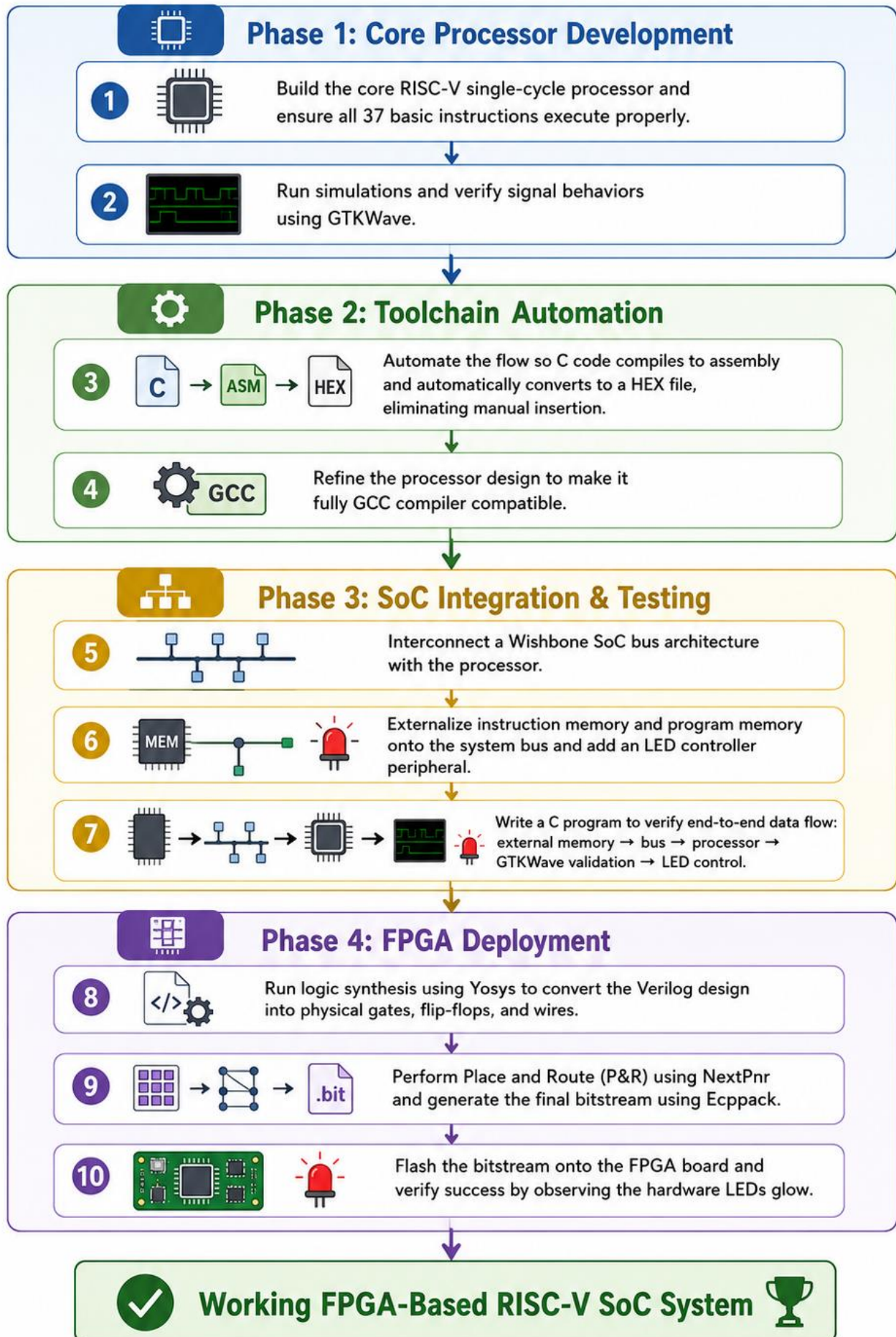
4 Design Methodology

4.1 Overall Development Strategy

The project follows a staged hardware/software co-design methodology. First, a hardware-only LED blinker is used to validate the board, clock, LED pins, bitstream generation, and programming command. Second, the full Wishbone-based processor system is built and tested. This staged method is important because it prevents board-level issues from being confused with CPU-level or firmware-level issues.

The CA project folder explicitly separates these two phases. The `hard_blinker` folder contains a minimal counter-driven LED design. The `wishbone_fixed` folder contains the patched CPU, Wishbone bus, firmware, ROM initialization files, constraints, and build scripts. The report is mainly based on the `wishbone_fixed` folder because it represents the final processor-based implementation.

Computer Architecture Implementation Path



4.2 Design Flow

The development flow is shown in Figure 1. The process begins with firmware and RTL preparation, followed by simulation, synthesis, placement, routing, bitstream generation, and board programming.

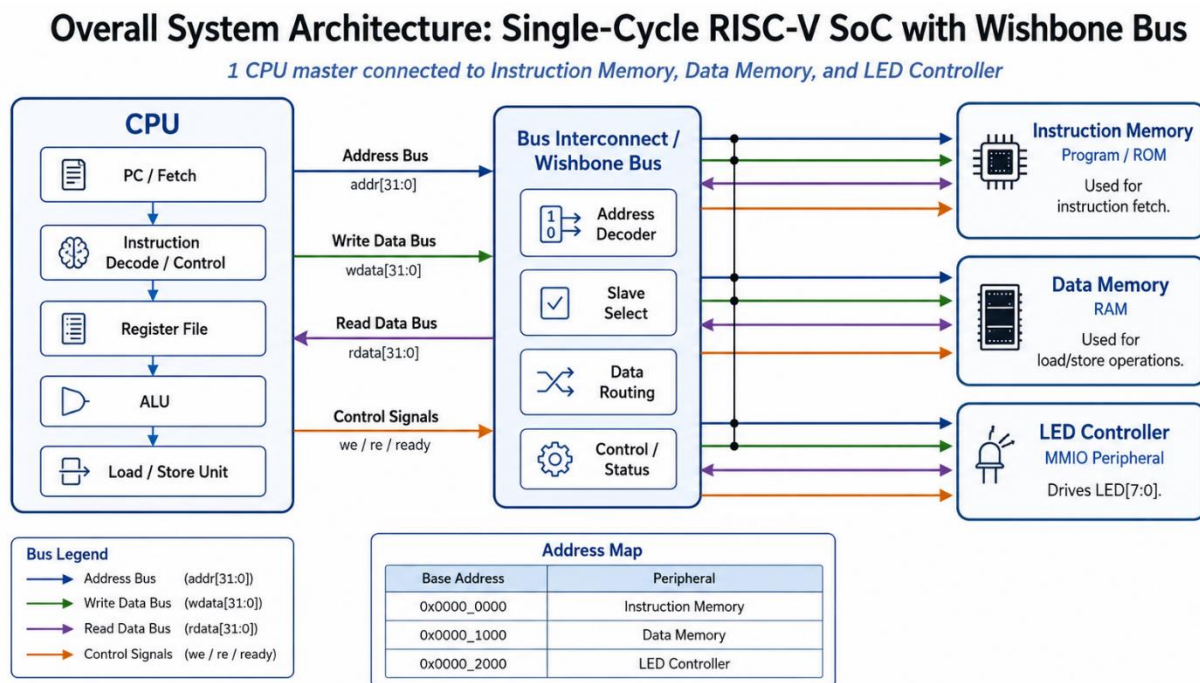


Figure 1: Design methodology flow from Verilog and C firmware to FPGA LED output.

4.3 Hardware/Software Boundary

The boundary between hardware and software is the memory map. The firmware does not directly manipulate FPGA pins. Instead, it writes a 32-bit word to a memory-mapped address. The hardware interconnect decodes that address and selects the LED slave. This is the same design principle used in real microcontroller systems, where software writes to peripheral registers rather than driving pins directly.

In the submitted firmware, the LED address is defined as:

```
#define LED_ADDR ((volatile unsigned int *)0x00002000)
```

Listing 1: Memory-mapped LED address in the submitted firmware.

The volatile qualifier is necessary because the address represents hardware I/O. It prevents the compiler from optimizing away repeated writes to the same address.

4.4 Verification Method

The verification method has three levels:

1. **Board sanity verification:** run the hard blinker to check the clock and LED pins.
2. **Functional simulation:** compile the Verilog testbench, generate a VCD file, and inspect internal

signals in GTKWave.

3. **Hardware verification:** synthesize, place, route, pack, and program the full SoC bitstream; then observe the LED pattern physically.

4.5 Reason for Keeping Screenshot Placeholders

The final screenshots are expected to include terminal build outputs, waveform windows, FPGA programming logs, and hardware photographs. Since those images may be generated after the report text is finalized, the report uses placeholders. This allows the document to remain compile-ready in Overleaf while leaving clear image positions for final evidence.

5 System Architecture

5.1 Top-Level System Description

The top-level architecture consists of an RV32I CPU, a Wishbone interconnect, a Wishbone ROM, a Wishbone RAM, and a Wishbone LED slave. The CPU is the only bus master. The three bus slaves occupy separate address ranges. During instruction fetch, the CPU sends a Wishbone read transaction to the ROM region. During load/store operations, the CPU sends a Wishbone transaction to either RAM or the LED peripheral depending on the computed address.

5.2 Top-Level SoC Block Diagram

Figure 2 is reserved for the final top-level block diagram. The figure should show the CPU as Wishbone master, the interconnect/address decoder, and the three slaves.

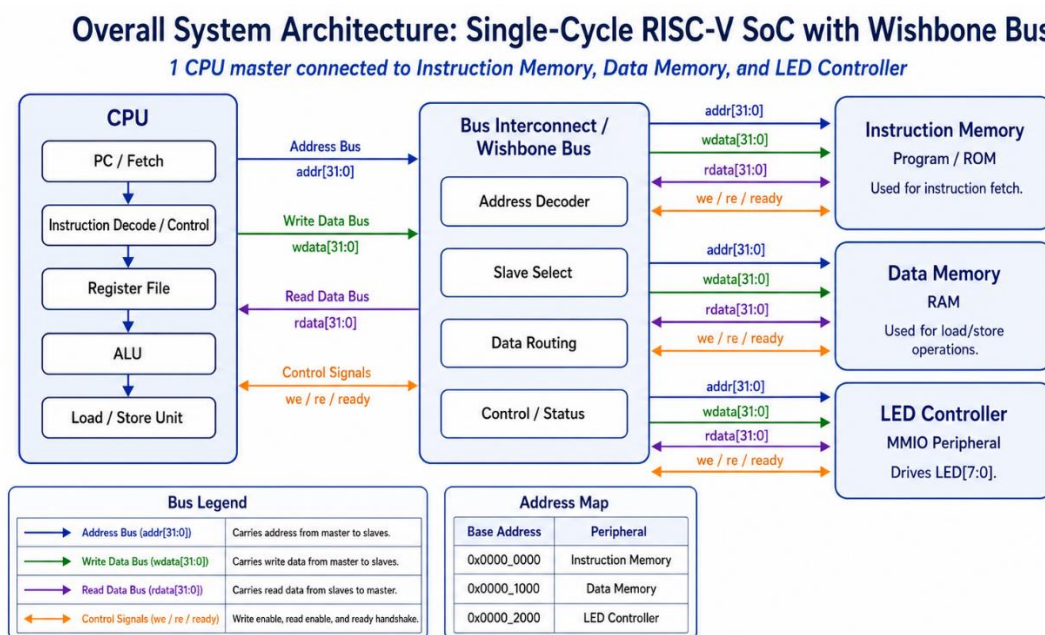


Figure 2: Overall SoC system block diagram showing CPU, Wishbone interconnect, ROM, RAM,

and LED peripheral.

5.3 Implemented Module Summary

The implemented modules and their responsibilities are summarized in Table 2. The file names are taken from the final fixed CA project folder.

Table 2: Main modules in the submitted Wishbone-based SoC.

File / module	Role	Important function in system
top.v / tope	Top-level CPU and SoC integration	Holds CPU FSM, PC, instruction register, Wishbone master logic, and slave instantiations.
ALU.v	Arithmetic and logic unit	Performs add, subtract, bit-wise, shift, set-less-than, and branch comparison operations.
ALU_control.v	ALU operation decoder	Converts opcode/function combinations into internal ALU control codes.
control_unit.v	Main control unit	Generates branch, memory, ALU source, register write, and jump control signals from opcode.
Register_file.v	Register file	Implements 32 registers with reset, two read ports, and one write port.
Immediate_Generator.v	Immediate generator	Extracts and sign-extends immediates for I, S, B, U, and J instruction types.
wb_interconnect.v	Wishbone interconnect	Decodes address regions and multiplexes slave acknowledgements/data back to the CPU.
wb_rom.v	Program ROM	Stores firmware instruction words initialized by rom_init.vh.
wb_ram.v	Data RAM	Provides byte-selectable read-/write data memory.
wb_led.v	LED peripheral	Captures write data and drives lower four LED outputs.

5.4 Data Flow

The data flow begins when the CPU fetches an instruction from ROM. The instruction is decoded into opcode, destination register, source registers, function fields, and immediate fields. The control unit and ALU control determine the required datapath operation. For ordinary ALU instructions, the result is written back to the register file. For store instructions, the ALU computes the target address, the store alignment logic prepares byte enable signals, and the Wishbone master initiates a write transaction. If the address belongs to the LED region, the interconnect selects the LED slave and the LED register updates.

5.5 Address Regions

The interconnect selects slaves using `m_adr_i[31:12]`. Table 3 shows the resulting memory map.

Table 3: Implemented address regions in the Wishbone interconnect.

Region	Address range	Slave	Access type
Program ROM	0x00000000–0x00000FFF	wb_rom.v	Read
Data RAM	0x00001000–0x00001FFF	wb_ram.v	Read/Write
LED peripheral	0x00002000–0x00002FFF	wb_led.v	Write/readback register
Default/unmapped	Other addresses	None	Acknowledged with zero data

6 RV32I Processor Core Design

6.1 Processor Organization

The processor core is implemented inside the top-level module `tope` in `top.v`. It contains a program counter, instruction register, instruction field extraction logic, control unit, immediate generator, register file, ALU control, ALU, branch/jump logic, load/store alignment logic, write-back selection, and a simple finite-state controller. The processor uses the Wishbone bus for both instruction fetch and data access.

6.2 Instruction Field Extraction

The instruction register stores the 32-bit instruction read from the program ROM. The fields are extracted directly using Verilog bit slicing:

- `opcode = instruction[6:0]`

- $rd = \text{instruction}[11:7]$
- $\text{funct3} = \text{instruction}[14:12]$
- $rs1 = \text{instruction}[19:15]$
- $rs2 = \text{instruction}[24:20]$
- $\text{funct7} = \text{instruction}[31:25]$

This field extraction matches the RV32I fixed-format instruction encoding and allows the control unit, register file, immediate generator, and ALU control block to operate independently.

6.3 Control Unit

The main control unit receives the 7-bit opcode and generates the data path control signals. The instructions format is shown below.

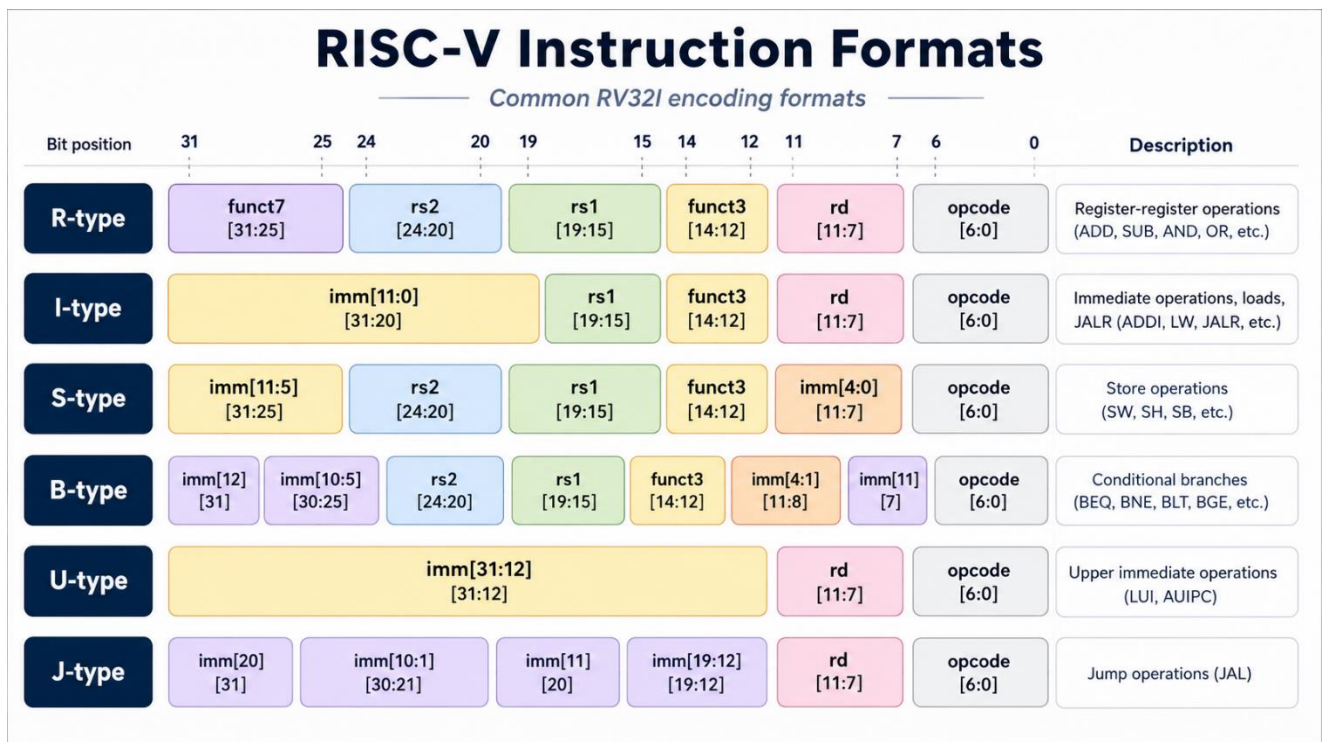


Figure 3: RISC-V instruction Formats

Table 4 summarizes the instruction classes implemented in the submitted control_unit.v file.

Table 4: Instruction classes decoded by the main control unit.

Opcode	Instruction class	Main action	Implemented
0110011	R-type	Register-register ALU operation	Yes
0010011	I-type ALU	Register-immediate ALU operation	Yes
0000011	Load	Memory read to register	Yes
0100011	Store	Register write to memory/peripheral	Yes
1100011	Branch	Conditional PC update	Yes
1101111	JAL	Jump and link	Yes
1100111	JALR	Register-indirect jump	Yes
0110111	LUI	Load upper immediate	Yes
0010111	AUIPC	Add upper immediate to PC	Yes

6.4 Immediate Generator

The immediate generator supports I-type, S-type, B-type, U-type, and J-type immediate formats. This is necessary because the firmware and compiled C program require immediate constants, store offsets, branch offsets, jump offsets, and upper immediate values. The generator sign-extends immediate fields where required and inserts the least-significant zero bit for branch and jump offsets.

6.5 Register File

The register file contains 32 registers and follows the normal RV32I rule that register x0 always reads as zero. The submitted fixed version includes reset propagation to the register file, ensuring that registers initialize to zero after FPGA configuration and startup reset. This is important because simulation may mask uninitialized values that become hardware startup problems after synthesis.

6.6 ALU and ALU Control

The ALU supports arithmetic, bitwise, shift, comparison, and branch comparison operations. The submitted `ALU_control.v` maps opcode/function information to internal 4-bit ALU operation codes. The operations include ADD, SUB, XOR, OR, AND, SLL, SRL, SRA, SLT, SLTU, and branch comparisons such as BEQ, BNE, BLT, BGE, BLTU, and BGEU.

6.7 Branch and Jump Logic

The branch target is computed as $pc + imm_ext$. The JALR target is computed as $(read_data1 + imm_ext) \& 32'hFFFF_FFFE$, which clears the least significant bit as required by the RISC-V indirect jump rule. LUI writes the immediate value directly, while AUIPC writes $pc + imm_ext$. The next PC selection therefore depends on branch result, jump control, JALR detection, and normal sequential execution.

6.8 Processor State Machine

The top-level CPU controller uses four states. The state behavior is summarized in Table 5.

Table 5: Processor finite-state behavior in the submitted top-level module.

State	Purpose	Exit condition
S_FETCH	Start an instruction fetch cycle	Move to S_WAIT_INSTR
S_WAIT_INSTR	Wait for ROM acknowledgement and capture instruction	wb_m_ack = 1
S_EXECUTE	Execute ALU, branch, jump, and non-memory operations	Move to data wait for memory operation, otherwise update PC
S_WAIT_DATA	Wait for load/store acknowledgement	wb_m_ack = 1

RISC-V Processor Internal Datapath and Finite-State Control Diagram

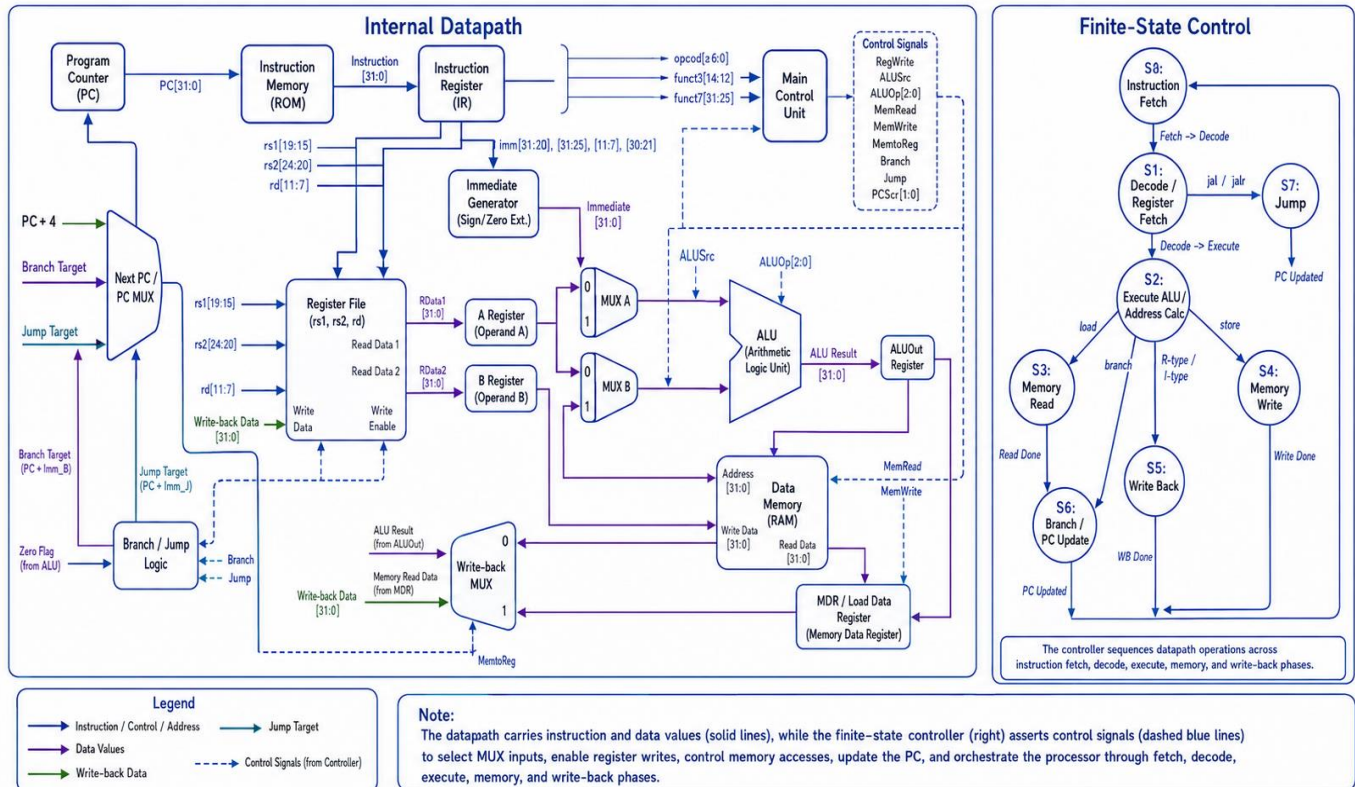


Figure 3: RISC-V processor internal datapath and finite-state control diagram.

6.9 Load and Store Alignment

The processor contains explicit store byte-enable logic and load extension logic. Store operations generate byte select signals for byte, halfword, and word stores. Load operations select and sign-extend or zero-extend bytes and halfwords according to `funct3`. This is a stronger implementation than a word-only memory system because it supports more of the RV32I load/store behavior.

7 Wishbone Interconnect and Memory Map

7.1 Wishbone Master Interface

The CPU generates Wishbone master signals from the current processor state. During instruction fetch, the bus address is the program counter. During data access, the bus address is the ALU result. Write enable is asserted only during store transactions in the data wait state. The bus cycle and strobe signals are asserted during instruction wait and data wait states.

7.2 Address Decoder

The `wb_interconnect.v` file decodes the selected slave using the upper address bits. The selection rules are:

$$\text{sel_s0} = \text{access} \ \&\& \ (\text{m_adr_i}[31:12] = 20'h00000), \quad (1)$$

$$\text{sel_s1} = \text{access} \ \&\& \ (\text{m_adr_i}[31:12] = 20'h00001), \quad (2)$$

$$\text{sel_s2} = \text{access} \ \&\& \ (\text{m_adr_i}[31:12] = 20'h00002). \quad (3)$$

This means that each slave occupies a 4 KB page in the memory map.

7.3 Read Data Multiplexing and Acknowledgement

The interconnect returns data and acknowledgement from only the selected slave. If no slave is selected but a bus access occurs, the interconnect returns zero data and acknowledges the transaction. This avoids a permanent bus stall on unmapped addresses, although future designs could replace this with a fault or trap mechanism.

7.4 Program ROM

The program ROM is implemented in `wb_rom.v`. It initializes all words to the RISC-V NOP instruction `0x00000013` and then includes `rom_init.vh`, which assigns compiled firmware words to ROM entries. The ROM acknowledges transactions synchronously and outputs the addressed word using `adr_i[9:2]`, which converts byte addresses into word indexes.

7.5 Data RAM

The RAM is implemented in `wb_ram.v`. It supports byte enables through the Wishbone `sel_i` bus. On a write transaction, each selected byte lane is updated independently. This supports store-byte, store-halfword, and store-word behavior when combined with the CPU store alignment logic.

7.6 LED Peripheral Region

The LED peripheral is selected when the address falls in the 0x00002000 page. The C program writes to 0x00002000, so `sel_s2` becomes active and the interconnect routes the transaction to `wb_led.v`. The lower four bits of the write data become the LED output value.

LED Peripheral Region

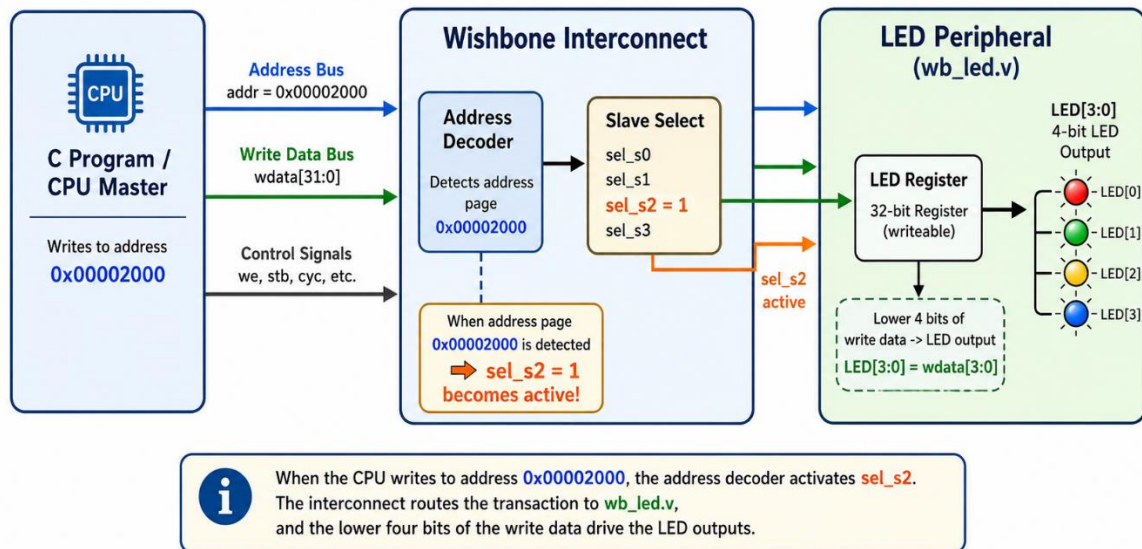


Figure 4: Wishbone bus architecture diagram with address decoder and LED slave.

7.7 Memory Layout in Linker Script

The linker script places executable code in ROM and data/BSS sections in RAM. The important memory declarations are:

```

1 MEMORY
2 {
3     ROM (rx) : ORIGIN = 0x00000000 , LENGTH = 4K
4     RAM (rwx) : ORIGIN = 0x00001000 , LENGTH = 4K
5 }
```

Listing 2: Memory regions from the submitted linker script.

This matches the Wishbone memory map and ensures that the reset entry point is located at the instruction memory base address.

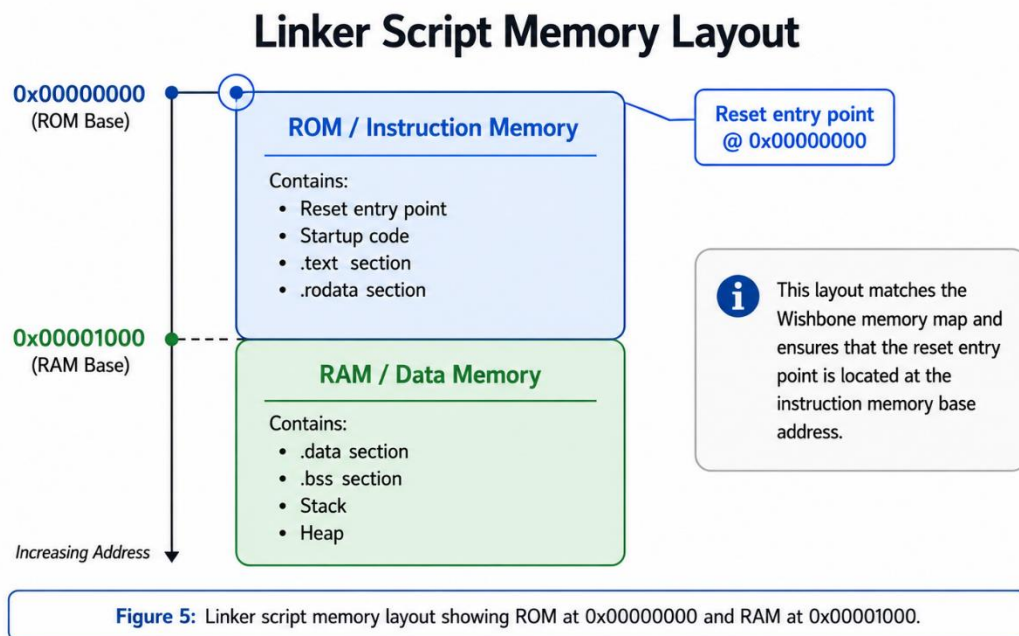


Figure 5: Linker script memory layout showing ROM at 0x00000000 and RAM at 0x00001000.

8 Firmware and ROM Initialization Flow

8.1 Bare-Metal Firmware Purpose

The firmware is responsible for producing a visible LED pattern by writing to the memory-mapped LED register. The submitted code.c file uses a volatile pointer at 0x00002000. The delay count was changed to 2000000u, which is much larger than the original value and is intended to make the LED changes visible to the human eye on FPGA hardware.

8.2 LED Pattern Sequence

The implemented firmware writes the following repeated sequence:

0x01 -> 0x02 -> 0x04 -> 0x08 -> 0x04 -> 0x02 -> repeat

This creates a moving LED pattern on four output pins. The firmware loop is shown in Listing 3.

```
1
2 while (1) {
3     led_write(0x01);
4     delay();
5     led_write(0x02);
6     delay();
7     led_write(0x04);
8     delay();
9     led_write(0x08);
10    delay();
11    led_write(0x04);
12    delay();
13    led_write(0x02);
14    delay();
15 }
```

Listing 3: LED pattern loop from the submitted firmware.

8.3 Startup Code

The startup file places the `__start` symbol in the `.text.start` section. It initializes the stack pointer using `_stack_top`, calls `main`, and then remains in an infinite loop if `main` returns. This is suitable for a bare-metal embedded program without an operating system.

8.4 Compiler and Linker Flow

The firmware build script uses `riscv64-unknown-elf-gcc` with RV32I-specific options. The key options include `-march=rv32i`, `-mabi=ilp32`, `-nostdlib`, `-nostartfiles`, `-ffreestanding`, `-fno-pic`, and `-mmodel=medlow`. These options keep the program compatible with the simple custom CPU and avoid dependencies on a standard runtime library. RISC-V-specific GCC options are documented by the GNU toolchain documentation [10].

8.5 Binary, HEX, and ROM Include Generation

After compilation, the ELF file is disassembled into `firmware.dump`, converted into a raw binary using `objcopy`, converted into little-endian 32-bit hex words using Python, and finally converted into `rom_init.vh`. The flow is summarized in Figure 6.

Binary, HEX, and ROM Include Generation

GCC compilation flow from C/startup/linker files to ELF, BIN, HEX, and ROM initialization include file

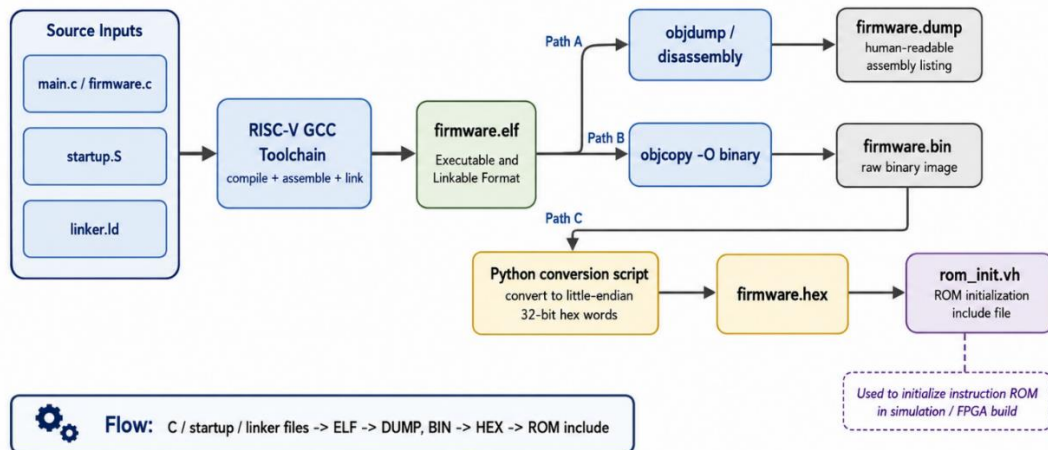


Figure 6: GCC compilation flow from C/startup/linker files to ELF, BIN, HEX, and ROM initialization include file.

```

(OSS CAD Suite) hassan516@DESKTOP-232RLFD:~/CEP_FPGA_LED_FIXED/wishbone_fixed$
riscv64-unknown-elf-gcc \
-march=rv32i \
-mabi=ilp32 \
-nostdlib \
-nostartfiles \
-ffreestanding \
-fno-builtin \
-O0 \
-Wl,--build-id=none \
-T link.ld \
startup.s code.c \
-o program.elf

riscv64-unknown-elf-objdump -D program.elf > firmware.dump

grep -i "2000" firmware.dump
grep -i "sw" firmware.dump

riscv64-unknown-elf-objcopy -O binary program.elf program.bin

xxd -p -c 4 program.bin | awk '{print substr($0,7,2) substr($0,5,2) substr($0,3,2) substr($0,1,2)}' > program.hex

head program.hex
wc -l program.hex

riscv64-unknown-elf-objcopy -O binary program.elf program.bin

xxd -p -c 4 program.bin | awk '{print substr($0,7,2) substr($0,5,2) substr($0,3,2) substr($0,1,2)}' > program.hex

head program.hex
wc -l program.hex
28: 00e7a023      SW      a4,0(a5) # 2000 <_stack_top>
14: 00812e23      SW      s0,28(sp)
1c: fea42623      SW      a0,-20(s0)
28: 00e7a023      SW      a4,0(a5) # 2000 <_stack_top>
40: 00812e23      SW      s0,28(sp)
48: fe042623      SW      zero,-20(s0)
58: fef42623      SW      a5,-20(s0)
80: 00112623      SW      ra,12(sp)
84: 00812423      SW      s0,8(sp)
00002117
00010113
074000ef
0000006f
fe010113
00812e23
02010413
fea42623
000027b7
fec42703
54 program.hex
  
```

Figure 7: Terminal screenshot showing successful firmware compilation and generated program.hex / rom_init.v

8.6 Hand-Assembled Firmware Reference

The project also includes `program_hand_assembled.S`, which explains the intended LED behavior in assembly form. This file is useful for understanding the machine-level operations required for LED control. It uses `lui` to load the LED base address, `sw` to write the current pattern, a decrementing delay loop, a shift-left operation to move the LED bit, and branch/jump instructions to repeat the sequence.

```
initial begin
    for (i = 0; i < 256; i = i + 1)
        rom[i] = 32'h00000013;    // NOP

    $readmemh("program.hex", rom);
end
```

Figure 8: the generated `program.hex` loaded into the Verilog ROM initialization process.

9 Memory-Mapped LED Peripheral

9.1 LED Slave Architecture

The LED peripheral is a Wishbone slave implemented in `wb_led.v`. It receives the clock, reset, address, write data, read data, write enable, byte select, cycle, strobe, and acknowledgement signals. Internally, it contains a 4-bit LED register. When a valid Wishbone write transaction arrives, the lower four bits of the write data are captured into this register. The register output is connected directly to the top-level `led[3:0]` output.

9.2 Wishbone Transaction Behavior

The LED slave asserts acknowledgement when both `cyc_i` and `stb_i` are active. This keeps the LED peripheral simple and gives the CPU a clear transaction completion signal. The read data output returns the current LED register value zero-extended to 32 bits. This is useful for debugging because the CPU or simulation waveform can observe the current LED register value if required.

9.3 LED Register Logic

The essential LED logic is shown in Listing 4. This block resets the LED register to zero and updates the register on valid write transactions.

```
1 always @(posedge clk) begin
2     if (reset) begin
3         led_reg <= 4'b0000;
4     end
5     else begin
```

```

6      if (cyc_i && stb_i && we_i) begin
7          led_reg <= dat_i[3:0];
8      end
9  end
10 end

```

Listing 4: LED register update logic from the submitted LED slave.

9.4 Software-to-Hardware LED Path

The complete path from C firmware to physical LED pins is:

1. Firmware writes a pattern to *LED_ADDR.
2. The compiler generates RV32I store instructions.
3. The CPU executes the store instruction and computes address 0x00002000.
4. The Wishbone master sends the address and data to the interconnect.
5. The interconnect selects slave S2 because $m_adr_i[31:12] = 20'h00002$.
6. The LED slave captures $dat_i[3:0]$.
7. The top-level LED outputs change on the FPGA pins.

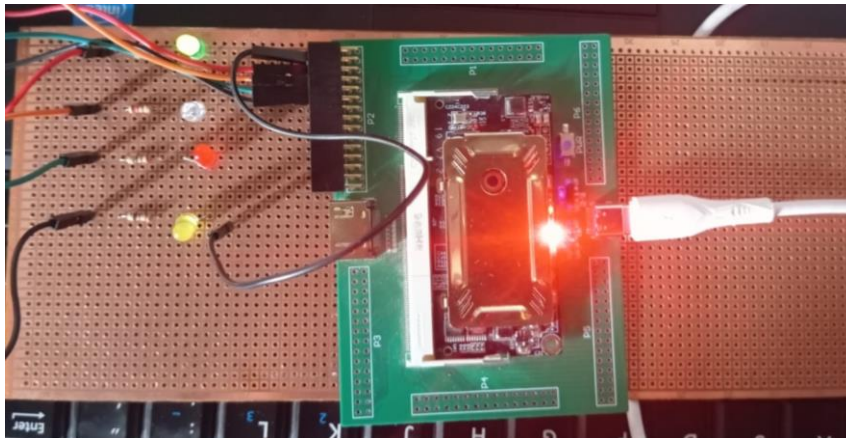


Figure 9: LED pattern flow diagram from C firmware to Wishbone write transaction and physical LED output pins.

9.5 Human-Visible Delay

A major hardware-related correction in the submitted fixed project was the delay value. A very small delay can look correct in simulation because waveform viewers can zoom into fast transitions, but the same delay is too fast for a human to see on FPGA LEDs. The

corrected firmware uses `DELAY_COUNT = 2000000u`. This correction is essential for physical demonstration.

9.6 Hardware-Only Blinker as First Test

The project also includes a hardware-only blinker. It uses a counter and connects counter bits directly to LEDs. This design does not depend on the CPU, ROM, RAM, Wishbone bus, C compiler, or reset path. Therefore, it is the safest first test for confirming that the FPGA board, clock, bitstream upload, and LED pin mappings are correct before debugging the full SoC.

10 Simulation and Verification

10.1 Verification Strategy

The verification strategy combines directed simulation with waveform analysis. Since the design contains a processor, memory, interconnect, and peripheral, a simple output-only test is not enough. Important internal signals must be observed to confirm that the CPU is fetching instructions, decoding them, performing ALU operations, issuing Wishbone transactions, receiving acknowledgements, and updating the LED register.

10.2 Testbench Role

The testbench should instantiate the top-level SoC, generate a clock, apply reset, and run the simulation for enough time to observe multiple firmware operations. The VCD dump file should include PC, instruction, state, Wishbone address, write data, acknowledgement, LED output, and selected register or ALU signals. The old report structure included a testbench section, and the current report keeps a placeholder for the testbench diagram.

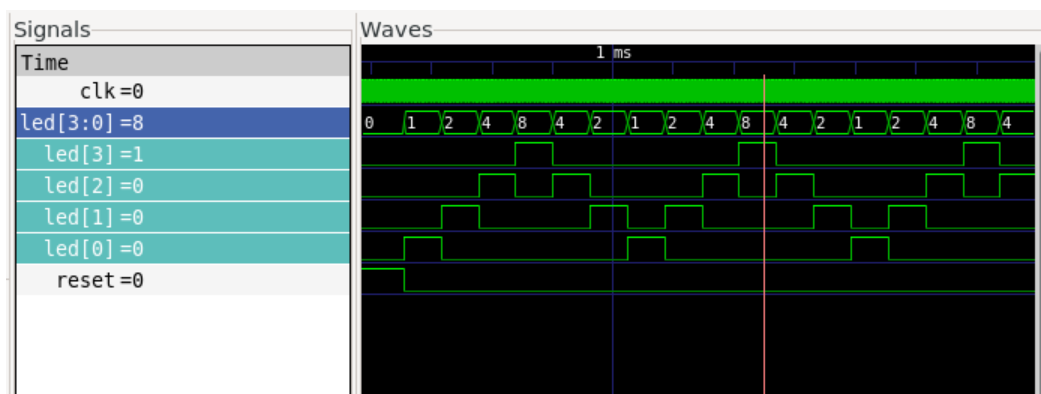


Figure 10: Testbench architecture showing clock/reset generation, SoC instance, waveform dump, and observed output signals.

10.3 Waveform Inspection in GTKWave

After simulation, GTKWave is used to inspect the generated VCD waveform. The key waveform checks are listed in Table 6.

Table 6: Functional verification checklist for waveform inspection.

Signal / behavior	Expected observation	Reason
Program counter	Starts at 0x00000000 and advances through firmware	Confirms correct reset and fetch sequence
Instruction register	Captures nonzero firmware words from ROM	Confirms ROM initialization and fetch acknowledgement
CPU state	Moves through fetch, wait, execute, and data access states	Confirms controller operation
Wishbone address	Shows ROM addresses during fetch and 0x00002000 during LED writes	Confirms memory map behavior
Wishbone acknowledgement	Asserted by selected slave during transaction completion	Prevents CPU from stalling
Write data	Carries LED pattern values such as 0x01, 0x02, 0x04, and 0x08	Confirms firmware data path
LED output	Updates according to the firmware pattern	Confirms LED slave behavior

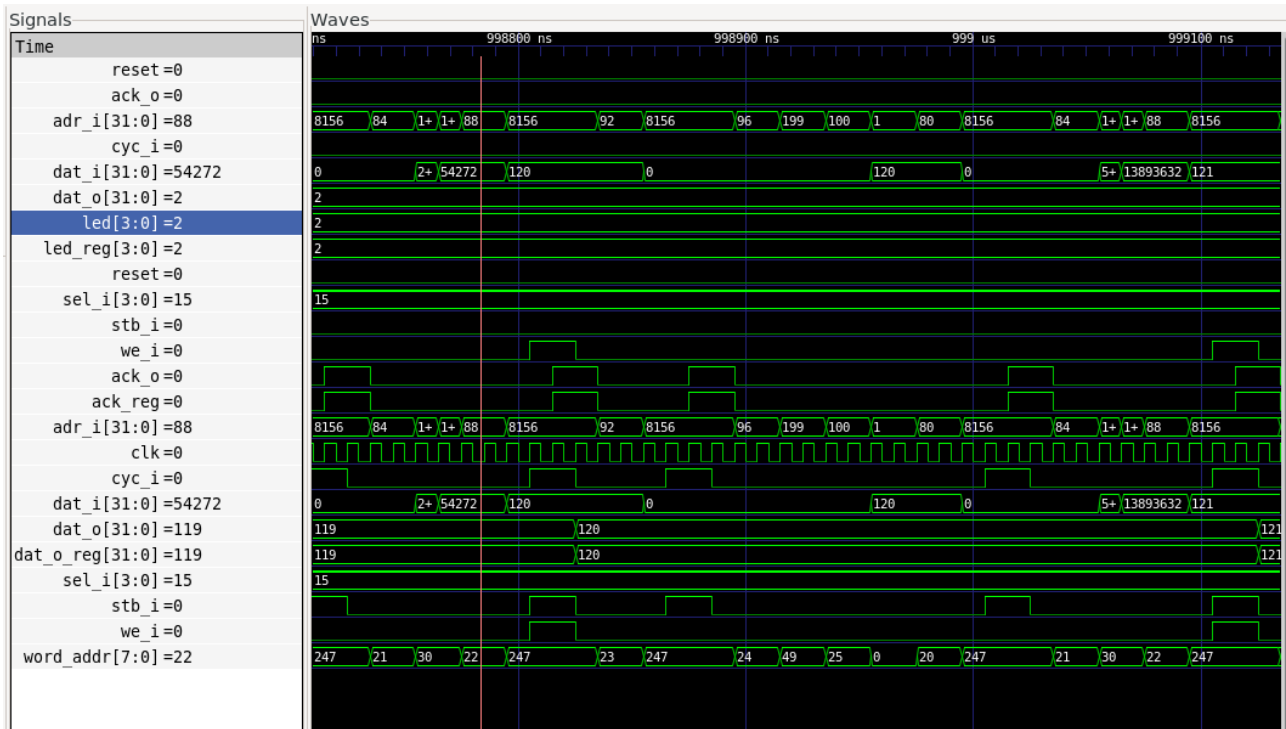


Figure 12: GTKWave screenshot showing PC, instruction, Wishbone address/data, acknowledgement, and LED output signals.

10.4 Simulation-to-Hardware Gap

Simulation can prove functional signal behavior, but it cannot fully prove physical FPGA behavior. Hardware execution can fail due to reset timing, clock frequency, pin constraints, bitstream loading issues, or LED delays that are too short for human observation. For this reason, the submitted fixed project includes specific hardware-oriented fixes and a hard blinker test.

11 FPGA Implementation Flow

11.1 Target FPGA and Board Assumptions

The submitted wishbone_fixed README describes a Colorlight i5 / Lattice ECP5 45K target flow. The constraint file constraints_colorlight_i5.lpf maps the clock to pin P3, reset to P18, and four LED outputs to pins K18, T18, R17, and M17. A separate ULX3S constraint file is also included for a board sanity test and alternate implementation.

11.2 Synthesis with Yosys

Yosys reads the Verilog RTL files and synthesizes the design for the ECP5 architecture using synth_ecp5. The submitted Colorlight build script uses the command shown in Listing 6.

```

yosys -l yosys.log -p "synth_ecp5 -top tope -json cpu.json" \
ALU.v \
ALU_control.v \
AND_gate.v \
OR_gate.v \
Multiplexer.v \
PC_plus_4_adder.v \
PC_plus_offset_adder.v \
program_counter.v \
control_unit.v \
Immediate_Generator.v \
Register_file.v \
wb_interconnect.v \
wb_rom.v \
wb_ram.v \
wb_led.v \
top.v

```

```

+-----Count including submodules.
|
4111 wires
10897 wire bits
4111 public wires
10897 public wire bits
3 ports
6 port bits
- memories
- memory bits
- processes
11 cells
9 $scopeinfo
2 DP16KD
6659 submodules
87 CCU2C
328 L6MUX21
4086 LUT4
953 PFUMX
1205 TRELIS_FF

```

Figure 14: Yosys synthesis and Look Up Tables

11.3 Place-and-Route with nextpnr-ecp5

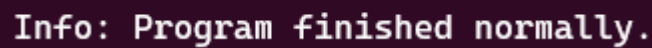
The nextpnr command uses the synthesized JSON file, package selection, LPF constraint file, and text configuration output. The submitted Colorlight build script uses `-45k -package CABGA381`, matching the intended ECP5 45K target flow.

```

1  nextpnr-ecp5 \
2    --45k \
3    --package CABGA381 \
4    --json cpu.json \
    --lpf constraints_colorlight_i5.lpf \
    --textcfg cpu.config

```

Listing 7: nextpnr-ecp5 place-and-route command from the submitted build script.



Info: Program finished normally.

Figure 15: nextpnr-ecp5 placement and routing.

11.4 Bitstream Packing with ecppack

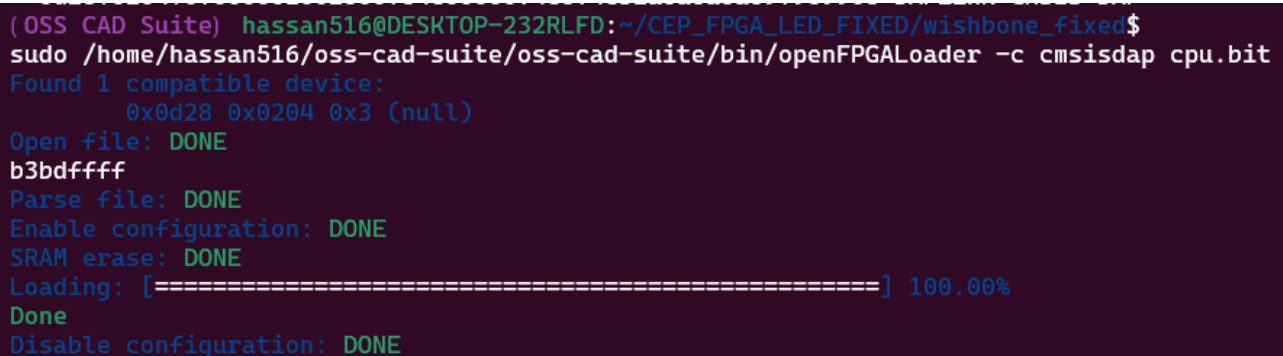
After place-and-route, ecppack converts `cpu.config` into `cpu.bit`. The submitted Colorlight script uses an explicit IDCODE:

```

1  ecppack --idcode 0x41112043 cpu.config cpu.bit

```

Listing 8: ecppack bitstream generation command.



```

(OSS CAD Suite) hassan516@DESKTOP-232RLFD:~/CEP_FPGA_LED_FIXED/wishbone_fixed$
sudo /home/hassan516/oss-cad-suite/oss-cad-suite/bin/openFPGALoader -c cmsisdap cpu.bit
Found 1 compatible device:
    0x0d28 0x0204 0x3 (null)
Open file: DONE
b3bdffff
Parse file: DONE
Enable configuration: DONE
SRAM erase: DONE
Loading: [=====] 100.00%
Done
Disable configuration: DONE

```

Figure 16: ecppack bitstream generation and `cpu.bit` file creation.

11.5 Programming with openFPGALoader

The final bitstream is programmed using openFPGALoader. The README indicates the CMSIS-DAP flow for the Colorlight i5 target:

```

1  sudo ~/fpga/openFPGALoader/build/openFPGALoader -c cmsisdap --detect
2  sudo ~/fpga/openFPGALoader/build/openFPGALoader -c cmsisdap cpu.bit

```

Listing 9: FPGA detection and programming commands.

11.6 Pin Constraint Summary

The important Colorlight i5 pin mappings are shown in Table 7. These entries are taken from the submitted `constraints_colorlight_i5.lpf` file.

Table 7: Pin assignments in the Colorlight i5 LPF file.

Top-level port	FPGA pin	I/O standard	Purpose
clk	P3	LVCNMOS33	25 MHz input clock
reset	P18	LVCNMOS33 with pulldown	Active-high reset input
led[0]	K18	LVCNMOS33	LED output bit 0
led[1]	T18	LVCNMOS33	LED output bit 1
led[2]	R17	LVCNMOS33	LED output bit 2
led[3]	M17	LVCNMOS33	LED output bit 3

```
LOCATE COMP "clk" SITE "P3";
IOBUF PORT "clk" PULLMODE=NONE IO_TYPE=LVCNMOS33;
FREQUENCY PORT "clk" 25 MHZ;

LOCATE COMP "reset" SITE "P18";
IOBUF PORT "reset" PULLMODE=DOWN IO_TYPE=LVCNMOS33 DRIVE=4;

LOCATE COMP "led[0]" SITE "K18";
LOCATE COMP "led[1]" SITE "T18";
LOCATE COMP "led[2]" SITE "R17";
LOCATE COMP "led[3]" SITE "M17";

IOBUF PORT "led[0]" PULLMODE=NONE IO_TYPE=LVCNMOS33 DRIVE=4;
IOBUF PORT "led[1]" PULLMODE=NONE IO_TYPE=LVCNMOS33 DRIVE=4;
IOBUF PORT "led[2]" PULLMODE=NONE IO_TYPE=LVCNMOS33 DRIVE=4;
IOBUF PORT "led[3]" PULLMODE=NONE IO_TYPE=LVCNMOS33 DRIVE=4;
```

Figure 18: LPF constraint file and final pin mapping used for the board.

12 Debugging and Hardware Fixes

12.1 Why Hardware Debugging Was Necessary

A design may appear correct in simulation but fail on FPGA hardware. This happens because simulation often starts registers with known values, ignores physical pin mistakes, and allows very short timing loops to be seen in a waveform viewer. FPGA hardware requires clean reset behavior, correct clock and pin constraints, visible human-scale LED timing, and reliable bus acknowledgement behavior.

The submitted fixed project files explicitly document several fixes that were required to move from simulation behavior to hardware-visible LED blinking. These fixes are summarized in Table 8.

Table 8: Important debugging fixes found in the submitted fixed CA project.

Issue	Observed or likely problem	Implemented correction
Firmware delay too small	Original delay was visible in simulation but too fast for physical LEDs	Changed firmware delay to <code>DELAY_COUNT = 2000000u</code>
Startup reset incomplete	Reset logic existed but was not consistently applied to CPU elements	Introduced startup reset and used <code>cpu_reset</code> throughout the SoC
ROM/RAM/LED acknowledgement startup	Unreset acknowledgement/data registers can create hardware-only startup problems	Added cleaner reset behavior for ROM, RAM, and LED-related registers

Issue	Observed or likely problem	Implemented correction
Register file reset ignored	Registers could start unknown after FPGA configuration	Added reset handling inside the register file
Constraint conflicts	Multiple constraint files could map pins inconsistently	Separated Colorlight and ULX3S constraint files
Board/pin uncertainty	Full CPU design is hard to debug if LED pins or clock are wrong	Added hardware-only blinker as first test

12.2 Startup Reset Correction

The submitted `top.v` includes a startup reset counter. After FPGA configuration, this counter keeps the CPU in reset until the counter reaches `16'hFFFF`. This gives internal registers a controlled initialization window. The design assigns `cpu_reset = startup_reset`, ensuring that the CPU begins from a known state even when the external reset is not actively pressed.

12.3 Register File Reset

The fixed register file resets all 32 registers to zero. Although register `x0` is always read as zero by design, the remaining registers still need deterministic startup behavior. Without reset, the firmware could begin with unknown register contents on hardware. The submitted fix is shown in the register file source and is included in the appendix.

12.4 Acknowledgement Handling

Wishbone acknowledgement behavior is critical because the CPU waits for `wb_m_ack` before progressing after instruction fetches and data transactions. If a slave never acknowledges, the CPU stalls. If a slave acknowledges incorrectly, the CPU may capture invalid data. The fixed ROM and RAM use registered acknowledgements, while the LED slave acknowledges valid cycle/strobe transactions directly.

12.5 Delay Loop Fix

The original delay value was reported as too small for hardware visibility. This is a common error in FPGA demonstration projects: a delay that is clear in GTKWave may be far too fast for physical observation. The corrected value in `code.c` makes the LED pattern human-visible when the FPGA clock is running at board speed.

12.6 Constraint File Separation

The fixed project separates board-specific constraints. The Colorlight i5 constraint file maps the design to the Colorlight pins, while the ULX3S file maps the design to onboard ULX3S LEDs. This prevents a situation where the design is logically correct but the wrong physical pins are driven.

13 Results and Discussion

13.1 Functional Result

The expected functional result is that the processor fetches firmware instructions from ROM, executes a delay-controlled LED sequence, writes LED values to address 0x00002000, and causes the FPGA LED outputs to follow the sequence 0001, 0010, 0100, 1000, 0100, 0010, and repeat. The report includes placeholders for the waveform evidence and physical FPGA photograph that should confirm this behavior.

13.2 Instruction Coverage

The final firmware does not require every RV32I instruction, but the processor design includes support for several important instruction classes. Table 9 summarizes the practical coverage relevant to the submitted design.

Table 9: Instruction behavior required or supported by the project.

Category	Examples	Purpose in system
Upper immediate	LUI, AUIPC	Load base addresses and PC-relative constants
Integer ALU	ADD, ADDI, SUB, logical operations	Perform arithmetic and counter updates
Shift and compare	SLLI, SLTIU, signed/unsigned compare	Move LED bit and test loop limits
Store	SW, plus byte/halfword support in store logic	Write data to RAM or LED peripheral

Load	LB, LH, LW, LBU, LHU logic	Read from memory when needed by compiled programs
Branch and jump	BNE, BEQ, JAL, JALR	Implement loops and program flow

13.3 Hardware Build Result

The submitted CA folder contains build scripts and a generated `cpu.bit` bitstream in the fixed Wishbone project. For final submission, the build terminal output should be inserted in the placeholders created in Chapter 11. The bitstream generation evidence should show successful completion of Yosys synthesis, nextpnr place-and-route, ecppack bitstream packing, and openFPGALoader programming.

13.4 Expected LED Output Evidence

The final hardware evidence should include a photograph or short frame sequence showing the FPGA LED pattern. Because the report is prepared before the final screenshots are inserted, Figure 20 is reserved for this proof.

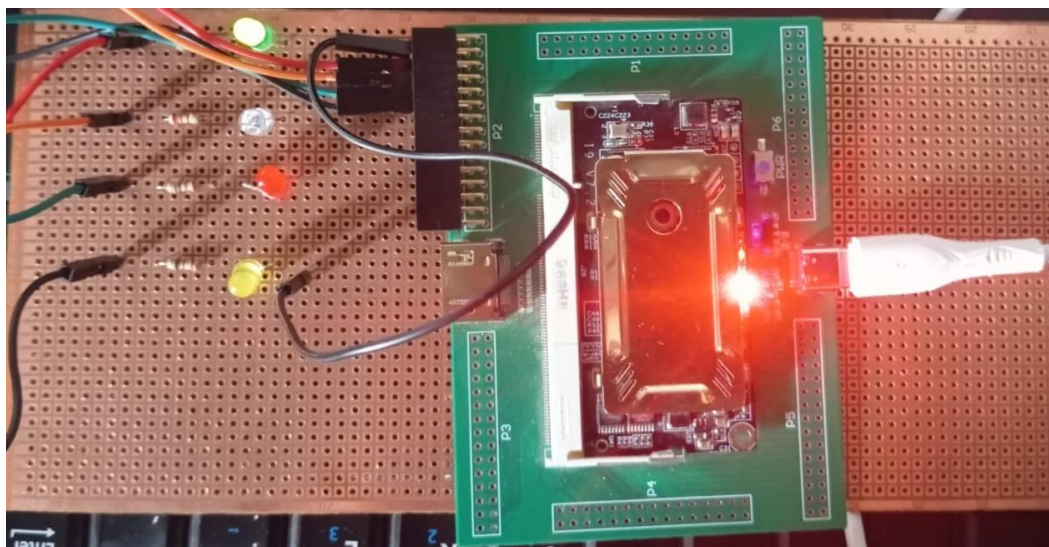


Figure 20: Physical FPGA board photograph showing the LED pattern generated by the RV32I processor firmware.

13.5 Discussion of Design Quality

The design is strong as a learning-oriented SoC because it contains all the main stages of a processor-driven embedded implementation. The memory map is simple but realistic. The Wishbone interface separates the CPU from peripherals. The firmware flow demonstrates how C code becomes ROM instructions. The FPGA build scripts document a practical implementation path. The debugging fixes also show awareness of hardware-specific issues, not just simulation correctness.

At the same time, the design remains intentionally simple. It does not include pipelining, in-terrupts, caches, privilege modes, exceptions, compressed instructions, multiplication/division, or a UART console. These omissions are acceptable for the course project because the goal is to demonstrate a complete CPU-to-peripheral path rather than implement a production microcontroller.

13.6 Resource Utilization

The final resource utilization should be copied from the synthesis or place-and-route report generated during the final build. A placeholder table is provided in Table 10.

Table 10: Resource utilization placeholder to be filled from final synthesis/place-and-route report.

Resource	Used	Available	Utilization
LUTs / logic cells	To be filled	To be filled	To be filled
Flip-flops	To be filled	To be filled	To be filled
Block RAM / EBR	To be filled	To be filled	To be filled
I/O pins	To be filled	To be filled	To be filled
Maximum frequency	To be filled	Not applicable	To be filled

14 Limitations and Future Improvements

14.1 Current Limitations

The current project is sufficient for demonstrating a processor-controlled memory-mapped LED peripheral, but it has limitations. The processor is not pipelined, so instruction throughput is limited. The design does not include exception handling, privilege modes, interrupts, or a standard debug interface. The memory sizes are small, and the ROM initialization method is simple. The LED output is useful for demonstration but does not provide rich runtime debugging.

The build flow is also board-specific. The Colorlight i5 and ULX3S pin constraints must not be mixed. If the design is moved to another ECP5 board, the LPF file must be changed according to the exact board schematic. Similarly, the programming command must match the physical

programmer interface connected to the board.

14.2 Possible Future Improvements

Future improvements can extend both the processor and the SoC. A UART peripheral could be added to provide serial output for debugging. A timer peripheral could generate accurate delays rather than using busy-wait loops. A seven-segment display or GPIO input peripheral could be mapped into another Wishbone address region. The CPU could be extended with exception support, a simple interrupt controller, or a pipelined datapath.

The verification environment could also be improved. A self-checking testbench would reduce dependence on manual waveform inspection. Instruction-level tests could be added to verify each supported RV32I instruction. Formal checks could be added for the Wishbone handshake and register file behavior. These improvements would make the design more reliable and easier to scale.

14.3 Recommended Next Version

The most practical next version of the project would add a UART transmitter mapped to a new address such as 0x00003000. The firmware could then print simple progress characters while also blinking LEDs. This would make hardware debugging much easier because the board could provide both visible LED output and serial text output.

15 Conclusion

This project demonstrates a complete Computer Architecture implementation flow from processor design to FPGA-visible output. A custom RV32I processor was integrated with a Wishbone interconnect, program ROM, data RAM, and a memory-mapped LED peripheral. The firmware writes LED pattern values to address 0x00002000, and the hardware interconnect routes those writes to the LED slave.

The project is valuable because it connects concepts that are often studied separately: instruction formats, control signals, ALU operations, memory maps, bus protocols, linker scripts, firmware compilation, ROM initialization, simulation waveforms, synthesis, constraints, bit-stream generation, and board programming. The fixed CA project files also show practical debugging lessons, especially the need for hardware-safe reset behavior, visible LED timing, correct acknowledgement handling, and board-specific pin constraints.

The final report has been prepared as an Overleaf-ready LaTeX project with placeholders for all screenshots and photographs. Once the final terminal outputs, GTKWave screenshots, and FPGA LED photographs are inserted, the document will provide a complete academic record of the design, verification, debugging, and implementation of the RV32I Wishbone LED SoC.

A Selected Source Code Listings

This appendix includes selected source files from the submitted fixed CA project. Large generated files such as `cpu.json`, `cpu.config`, VCD waveforms, and bitstreams were intentionally not included in the Overleaf project because they are not required for report compilation and would make the ZIP unnecessarily large.

A.1 Firmware Source

```

1  #define LED_ADDR ((volatile unsigned int *)0x00002000)
2
3  /* Original value was 5, which is visible in simulation but far too fast
4     for real LEDs.
5     This larger loop gives a human-visible delay after synthesis to FPGA
6     hardware. */
7
8  #define DELAY_COUNT 2000000
9     u
10
11  static void led_write(unsigned int pattern)
12  {
13      *LED_ADDR = pattern;
14  }
15
16  static void delay(void)
17  {
18      volatile unsigned int i;
19      for (i = 0; i < DELAY_COUNT; i++) {
20      }
21  }
22
23  void main(void)
24  {
25      while (1) {
26          led_write(0x01);
27          delay();
28          led_write(0x02);
29          delay();
30          led_write(0x04);
31          delay();
32          led_write(0x08);
33          delay();
34          led_write(0x04);
35          delay();
36          led_write(0x02);
37          delay();
38      }
39  }

```

Listing 10: Submitted C firmware for memory-mapped LED blinking

References

- [1] RISC-V International, “Ratified RISC-V Specifications,” online. Available: <https://riscv.org/specifications/ratified/>. Accessed: Jun. 2026.
- [2] RISC-V International, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” specification. Available: <https://github.com/riscv/riscv-isa-manual>. Accessed: Jun. 2026.
- [3] OpenCores, “WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores, Revision B.4,” 2010. Available: https://cdn.opencores.org/downloads/wbspec_b4.pdf. Accessed: Jun. 2026.
- [4] YosysHQ, “synth_ecp5 - Synthesis for ECP5 FPGAs,” online documentation. Available: https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/synth_ecp5.html. Accessed: Jun. 2026.
- [5] YosysHQ, “nextpnr: A portable FPGA place and route tool,” GitHub repository. Available: <https://github.com/YosysHQ/nextpnr>. Accessed: Jun. 2026.
- [6] YosysHQ, “Project Trellis: Lattice ECP5 bitstream documentation and tools,” GitHub repository. Available: <https://github.com/YosysHQ/prjtrellis>. Accessed: Jun. 2026.
- [7] trabucayre, “openFPGALoader: Universal utility for programming FPGAs,” GitHub repository. Available: <https://github.com/trabucayre/openFPGALoader>. Accessed: Jun. 2026.
- [8] Icarus Verilog Project, “Icarus Verilog Documentation,” online documentation. Available: <https://steveicarus.github.io/iverilog/>. Accessed: Jun. 2026.
- [9] GTKWave Project, “GTKWave Wave Viewer,” project documentation. Available: <https://gtkwave.sourceforge.net/>. Accessed: Jun. 2026.
- [10] GNU Project, “GCC RISC-V Options,” online documentation. Available: <https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html>. Accessed: Jun. 2026.